Chapter 07: Compound Types and References

ENG3091

Khiem Nguyen

Contents

1 Compound types **2**
- 1.1 Types built from other types . . . 2
- 1.2 The compound types of C++ . . . 2

2 Value categories: lvalues and rvalues **2**
- 2.1 Two properties of every expression . . . 2
- 2.2 Why is `x = 5` fine, but `5 = x` isn't? . . . 3
- 2.3 lvalues and rvalues . . . 3
- 2.4 Spot the rvalues . . . 4
- 2.5 lvalue-to-rvalue conversion . . . 4
- 2.6 A tool to tell them apart . . . 5

3 lvalue references **5**
- 3.1 A reference is another name for an object . . . 5
- 3.2 Your first reference . . . 6
- 3.3 Changing an object through a reference . . . 6
- 3.4 A reference must be initialized . . . 7
- 3.5 Which objects can a reference bind to? . . . 7
- 3.6 A reference can't be reseated . . . 8
- 3.7 Lifetimes and dangling references . . . 8
- 3.8 References are not objects . . . 9

4 Pass by reference **9**
- 4.1 Pass by value can be expensive . . . 9
- 4.2 Pass by reference . . . 9
- 4.3 Value versus reference, in a picture . . . 10
- 4.4 Don't believe me? Check the addresses . . . 11
- 4.5 Changing the caller's variable . . . 11
- 4.6 The swap that finally swaps . . . 12
- 4.7 Non-const reference parameters are picky . . . 12
- 4.8 Pass by const reference . . . 13
- 4.9 A const reference can't change its referent . . . 13
- 4.10 Value, reference or const reference? . . . 14

5 Return by reference **14**
- 5.1 Return by reference . . . 14
- 5.2 The object must outlive the function . . . 15
- 5.3 A use for return by reference . . . 15

6 Wrapping up **16**
- 6.1 Summary . . . 16
- 6.2 Keep learning . . . 16

1 Compound types

1.1 Types built from other types

Imagine a program that keeps track of employees:

```
int id1{42};
int id2{24};
int id3{162};

double salary1{42123.3};
double salary2{43784.5};
double salary3{39469.2};
// ...names, roles, job titles, and so on
```

With 100 employees, this simply doesn't scale. We need types that are **built from other types**: **compound types** (also called *composite types*).

1.2 The compound types of C++

Built into the language

- functions (Chapter 02)
- references (**today**)
- pointers (Chapter 08)
- C-style arrays

Defined by you, the programmer

- enumerations (`enum`, `enum class`)
- structs (`struct`)
- classes (`class`): the heart of this course, from Chapter 10
- unions (`union`)

We won't cover all of them: there isn't time. You'll see that structs and classes are very similar, so studying one carefully is enough.

Even a function has a compound type: the type of `void do_something(int x, double y)` is `void(int, double)`.

2 Value categories: lvalues and rvalues

2.1 Two properties of every expression

Recall: an **expression** combines literals, variables, operators and function calls to produce a value. Every expression has:

- a **type**: the type of the value it produces;

```
auto v1{12 / 4};   // int / int gives int
auto v2{12.0 / 4}; // double / int gives double
```

- a **value category**: what *kind* of thing it produces, an object or just a value.

The type tells the compiler whether an expression makes sense in a given place: `print("foo")` won't compile if `print` expects an `int`. The value category does the same job in a different way. Before references can make sense, we need value categories.

2.2 Why is `x = 5` fine, but `5 = x` isn't?

Listing 1 `lvalue_assignment.cpp`

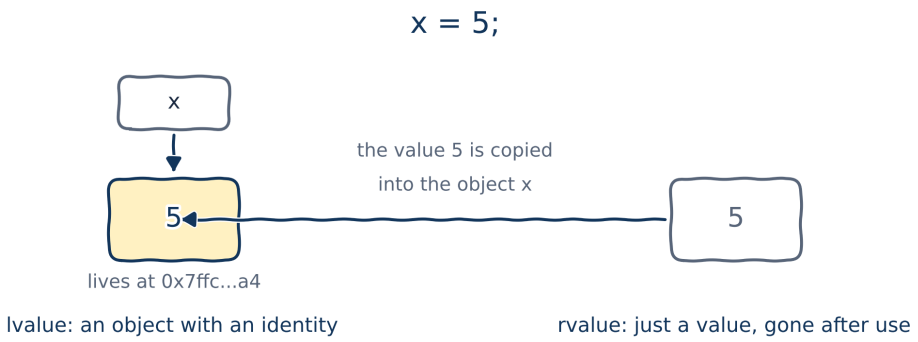
```cpp
int main() {
    int x{};
    x = 5; // fine: x is a variable we can assign to
    5 = x; // error: you can't assign to 5
    return 0;
}
```

```
error: lvalue required as left operand of assignment
```

As humans, we can see that `5 = x` makes no sense. But how does the compiler know which expressions may go on the left of `=`?

The answer is in the error message: the left side must be an **lvalue**.

2.3 lvalues and rvalues



- An **lvalue** evaluates to an **identifiable object** (or function): something with a name or an address, which outlives the expression. Example: a variable `x`.
- An **rvalue** evaluates to a **value**: a literal, a temporary result, something returned by value. It's gone at the end of the expression.

Since C++11 there are also *glvalues*, *prvalues* and *xvalues*, for a feature called *move semantics*. Lvalues and rvalues are enough for us.

2.4 Spot the rvalues

Listing 2 `rvalue_expression_example.cpp`

```cpp
int return5() { return 5; }

int main() {
    int x{5};             // 5 is an rvalue
    const double d{1.2}; // 1.2 is an rvalue

    int y{x};                    // x is a modifiable lvalue
    const double e{d};           // d is a non-modifiable lvalue
    int z{return5()};            // return5() is an rvalue (returned by value)
    int w{x + 1};                // x + 1 is an rvalue
    int q{static_cast<int>(d)}; // the converted value is an rvalue
    return y + z + w + q + static_cast<int>(e);
}
```

- `return5()`, `x + 1` and `static_cast<int>(d)` produce temporary values, not identifiable objects: they're rvalues.
- `static_cast<int>(d)` is an **explicit conversion**: it turns the `double` `1.2` into the `int` `1`.
- One surprise: a string literal like `"Hello"` is an lvalue. (It's stored somewhere in memory for the whole program.)

2.5 lvalue-to-rvalue conversion

Operators expect rvalues as operands, unless stated otherwise. But this compiles:

```cpp
int x{1};
int y{2};
x = y;     // y is an lvalue, not an rvalue!
x = x + 1; // x appears on both sides
```

- Where an rvalue is expected and an lvalue is given, the lvalue goes through an **lvalue-to-rvalue conversion**: it's simply evaluated to produce its value. In `x = y`, `y` becomes the value `2`.
- In `x = x + 1`, the left `x` is an lvalue (the object we assign to). The right `x` is converted to its value, say `42`, and `x + 1` produces the rvalue `43`.

> **!** Key idea
>
> An lvalue can be used wherever an rvalue is expected. An rvalue can never be used where an lvalue is required.

2.6 A tool to tell them apart

Listing 3 `tell_value_categories.cpp`

```cpp
#include "value_category.h" // our little tool
#include <string>

int get_int() { return 5; }

int main() {
    int x{5};
    PRINT_VALUE_CATEGORY(5);
    PRINT_VALUE_CATEGORY(get_int());
    PRINT_VALUE_CATEGORY(x);
    PRINT_VALUE_CATEGORY(x + 1);
    PRINT_VALUE_CATEGORY(std::string{"Hello"});
    PRINT_VALUE_CATEGORY("Hello");
    PRINT_VALUE_CATEGORY(++x);
    PRINT_VALUE_CATEGORY(x++);
    return 0;
}
```

```
5 is an rvalue
get_int() is an rvalue
x is an lvalue
x + 1 is an rvalue
std::string{"Hello"} is an rvalue
"Hello" is an lvalue
++x is an lvalue
x++ is an rvalue
```

The tool lives in `value_category.h`. Don't worry about how it works.

3 lvalue references

3.1 A reference is another name for an object

A **reference** is an **alias**: a second name for an object that already exists.

- Anything you do to the reference happens to the object it refers to.
- References may seem pointless at first. They're used *everywhere* in C++, as you'll see shortly.

```cpp
int        // an int
int&       // an lvalue reference to an int
double&    // an lvalue reference to a double
const int& // an lvalue reference to a const int
```

- `int&` (one ampersand) is an **lvalue reference**. There are also *rvalue references*, `int&&`, with two ampersands. They came with C++11, and many experienced programmers have never needed them.
- Since lvalue references are by far the most common, "reference" on its own means *lvalue reference*.

3.2 Your first reference

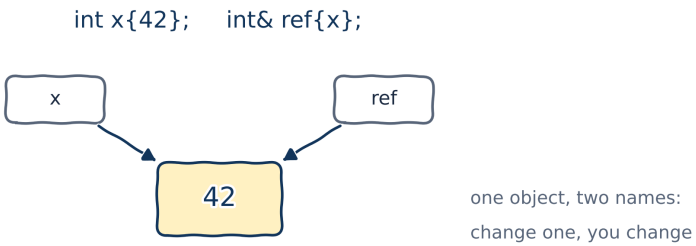
Listing 4 `lvalue_reference_variables.cpp`

```cpp
#include <iostream>

int main() {
    int x{5};    // an ordinary int variable
    int& ref{x}; // ref is a reference to x: another name for x

    std::cout << x << '\n';   // prints 5
    std::cout << ref << '\n'; // prints 5, through ref
    return 0;
}
```

```
5
5
```



3.3 Changing an object through a reference

Listing 5 `modify_value_through_nonconst_reference.cpp`

```cpp
#include <iostream>

int main() {
    int x{42};
    int& ref{x}; // ref is another name for x

    std::cout << x << ' ' << ref << '\n';

    x = 24; // change x...
    std::cout << x << ' ' << ref << '\n';

    ref = 12; // ...or change it through ref
    std::cout << x << ' ' << ref << '\n';
    return 0;
}
```

What will this print? Think first!

```
42 42
24 24
12 12
```

> Best practice
>
> Write the ampersand next to the type, `int& ref`, not next to the name. It's part of the type.

3.4 A reference must be initialized

Listing 6 `reference_must_be_initialized.cpp`

```cpp
int main() {
    int& invalid_ref; // error: a reference must be initialized

    int x{5};
    int& ref{x}; // fine: ref is bound to x
    return 0;
}
```

```
error: 'invalid_ref' declared as reference but not initialized
```

- Initializing a reference **binds** it to an object; the object is called its **referent**.
- Don't worry: I won't test your memory of these terms in the paper exam!

3.5 Which objects can a reference bind to?

Listing 7 `nonconst_reference_binding.cpp`

```cpp
int main() {
    int x{5};
    int& ref{x}; // fine: x is a modifiable lvalue

    const int y{5};
    int& invalid_ref{y}; // error: y is const

    int& invalid_ref2{0}; // error: 0 is an rvalue
    return 0;
}
```

```
error: binding reference of type 'int&' to 'const int' discards qualifiers
error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'
```

Why not? A non-const reference lets you **change** its referent. Binding one to a `const` object, or to a temporary value like `0`, would let you change something that must not, or cannot, change.

3.6 A reference can't be reseated

Listing 8 `reference_cannot_be_reseated.cpp`

```cpp
#include <iostream>

int main() {
    int x{5};
    int y{6};

    int& ref{x}; // ref is another name for x

    ref = y; // does this make ref refer to y?

    std::cout << x << '\n'; // what does this print?
    return 0;
}
```

```
6
```

`ref = y` does **not** make `ref` refer to `y`. `ref` *is* `x`, so this is simply `x = y`. Once bound, a reference stays bound to the same object for its whole life.

> **Coming from Python**
>
> A Python name can be rebound to a different object at any time: `a = b` does exactly that. A C++ reference never can be.

3.7 Lifetimes and dangling references

Listing 9 `lifetime_reference_and_referent.cpp`

```cpp
#include <iostream>

int main() {
    int x{42};

    {                           // a nested block
        int& ref{x};            // ref is a reference to x
        std::cout << ref << '\n'; // prints 42
    } // ref is destroyed here; x doesn't notice

    std::cout << x << '\n'; // still prints 42
    return 0;
} // x is destroyed here
```

```
42
42
```

- A reference and its referent have **independent lifetimes**. Destroying the reference doesn't affect the object.

- The other way round is dangerous: if the **object** is destroyed first, the reference is left **dangling**. Using a dangling reference is undefined behaviour.

3.8 References are not objects

- A reference is just a name. The compiler often replaces it with the referent and gives it no memory at all (though sometimes it does need some).
- So "reference variable" is a slight misnomer: variables are named *objects*.
- One consequence: you can't have a reference to a reference.

So far references look like a curiosity. Their real job is coming up next.

4 Pass by reference

4.1 Pass by value can be expensive

In Chapter 02 we learned **pass by value**: the argument is **copied** into the parameter. Copying a `double` is cheap; copying a long `std::string` is not:

Listing 10 `pass_by_value_string.cpp`

```cpp
#include <iostream>
#include <string>

void print_value(std::string y) { // y is a copy of x
    std::cout << y << '\n';
} // the copy is destroyed here

int main() {
    std::string x{"My name is Khiem. What's your name?"};
    print_value(x); // copies every character of x: expensive!
    return 0;
}
```

The copy is made, used once, and thrown away. A waste! Most standard library types, like `std::string`, are expensive to copy.

4.2 Pass by reference

Make the parameter a **reference**, and it is simply bound to the argument. No copy is made:

Listing 11 `pass_by_reference.cpp`

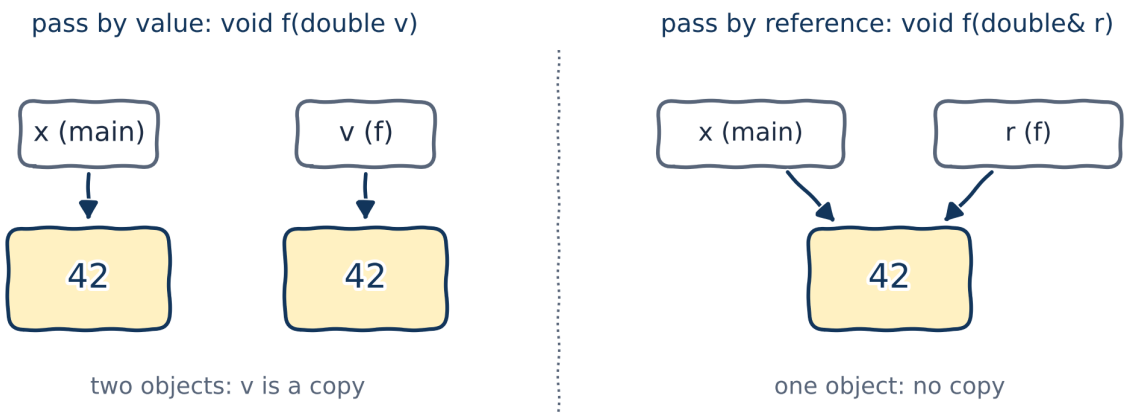
```cpp
#include <iostream>
#include <string>

void print_value(std::string& y) { // y is a reference: no copy
    std::cout << y << '\n';
}

int main() {
    std::string x{"Hello, world!"};
    print_value(x); // y is bound to x: cheap
    return 0;
}
```

```
Hello, world!
```

4.3 Value versus reference, in a picture



- **Pass by value:** the parameter is a new object, initialized with a copy of the argument.
- **Pass by reference:** the parameter is just another name for the argument. Nothing is copied, and the function works on the caller's object itself.

4.4 Don't believe me? Check the addresses

Listing 12 `pass_by_value_vs_pass_by_reference.cpp`

```cpp
#include <iostream>

void print_addresses(double val, double& ref) {
    std::cout << "address of val (a copy):  " << &val << '\n';
    std::cout << "address of ref (no copy): " << &ref << '\n';
}

int main() {
    double x{42.0};
    std::cout << "address of x:             " << &x << '\n';
    print_addresses(x, x);
    return 0;
}
```

```
address of x:             0x7ffcc2285eb0
address of val (a copy):  0x7ffcc2285e98
address of ref (no copy): 0x7ffcc2285eb0
```

`ref` has the **same address** as `x`: it is `x`. `val` lives somewhere else. (`&x` gives the address of `x`, more in Chapter 08. Your addresses will differ.)

4.5 Changing the caller's variable

Listing 13 `pass_by_reference_change_value.cpp`

```cpp
#include <iostream>

void add_value(double& y, double v) { // y is bound to the caller's x
    y += v;                           // so this changes x itself
}

int main() {
    double x{42.0};
    std::cout << "x = " << x << '\n';
    add_value(x, 24.0);
    std::cout << "x = " << x << '\n';
    return 0;
}
```

```
x = 42
x = 66
```

Any change made through a non-const reference parameter changes the caller's argument itself. That's useful when a function's whole purpose is to modify something.

4.6 The swap that finally swaps

Listing 14 `swap_by_reference.cpp`

```cpp
#include <iostream>

void swap_values(int& x, int& y) { // no copies
    int temp{x};
    x = y;
    y = temp;
}

int main() {
    int x{100};
    int y{200};
    swap_values(x, y);
    std::cout << "x = " << x << ", y = " << y << '\n';
    return 0;
}
```

```
x = 200, y = 100
```

Remember the swap that didn't swap in Chapter 02? With references, it finally works.

4.7 Non-const reference parameters are picky

Listing 15 `pass_by_reference_accept_modifiable_lvalues.cpp`

```cpp
#include <iostream>

void print_value(int& y) { // only accepts modifiable lvalues
    std::cout << y << '\n';
}

int main() {
    int x{5};
    print_value(x); // fine: x is a modifiable lvalue

    const int z{5};
    print_value(z); // error: z is const
    print_value(5); // error: 5 is an rvalue
    return 0;
}
```

```
error: binding reference of type 'int&' to 'const int' discards qualifiers
error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'
```

A non-const reference parameter only accepts **modifiable lvalues**: in practice, non-const variables. But often we just want to *read* an expensive argument without copying it, whatever it is.

4.8 Pass by const reference

Listing 16 `pass_by_const_reference.cpp`

```cpp
#include <iostream>

void print_value(const int& y) { // a const reference
    std::cout << y << '\n';
}

int main() {
    int x{5};
    print_value(x); // fine: a modifiable lvalue

    const int z{6};
    print_value(z); // fine: a const lvalue

    print_value(7); // fine: an rvalue (bound to a temporary)
    return 0;
}
```

```
5
6
7
```

A **const reference** binds to anything: modifiable lvalues, const lvalues and rvalues. That's exactly why C++ allows const references to bind to rvalues. Without it, you couldn't pass a literal like 7 to a function taking a reference.

4.9 A const reference can't change its referent

Listing 17 `const_reference_cannot_modify.cpp`

```cpp
void add_value(const double& ref, double value) {
    ref = ref + value; // error: we promised not to change ref
}

int main() {
    double x{1.0};
    add_value(x, 2.0);
    return 0;
}
```

```
error: assignment of read-only reference 'ref'
```

No copy **and** no accidental changes: we're greedy, and with const references we get both.

4.10 Value, reference or const reference?

Best practice

- **Fundamental types** (`int`, `double`, `char`, `bool`): pass by **value**. They're cheap to copy.
- **Class types** (`std::string`, `std::vector`, your own classes later): pass by **const reference**, so there's no copy.
- Pass by **non-const reference** only when the function is meant to change the argument.

For read-only text, C++17 also offers `std::string_view`, a cheap view of a string. Strings are the case Chapter 04 promised we'd come back to.

5 Return by reference

5.1 Return by reference

Returning by value **copies** the result back to the caller. Returning a **reference** avoids that copy:

Listing 18 `return_by_ref_academic.cpp`

```cpp
#include <iostream>
#include <string>

const std::string& get_program_name() { // returns a const reference
    // static: created once, lives until the program ends
    static const std::string program_name{"Calculator"};
    return program_name;
}

int main() {
    std::cout << "This program is named " << get_program_name() << '\n';
    return 0;
}
```

```
This program is named Calculator
```

A **static** local variable is created once and lives until the program ends, unlike an ordinary local variable. Here that's essential, as the next slide shows.

5.2 The object must outlive the function

Listing 19 `return_by_ref_lifetime.cpp`

```cpp
#include <iostream>
#include <string>

const std::string& get_program_name() {
    // not static: destroyed when the function returns
    const std::string program_name{"Calculator"};
    return program_name; // returns a reference to a dying object
}

int main() {
    std::cout << "This program is named " << get_program_name() << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 -Wall return_by_ref_lifetime.cpp
warning: reference to local variable 'program_name' returned
$ ./a.out
Segmentation fault
```

`program_name` is destroyed when the function returns, so the caller receives a **dangling reference**: undefined behaviour. On my machine, it crashed.

5.3 A use for return by reference

Listing 20 `return_by_ref_larger.cpp`

```cpp
#include <iostream>

int& larger(int& a, int& b) { // returns a reference to a or to b
    return (a > b) ? a : b;
}

int main() {
    int x{5};
    int y{8};

    larger(x, y) = 0; // resets whichever is larger: here, y
    std::cout << "x = " << x << ", y = " << y << '\n';
    return 0;
}
```

```
x = 5, y = 0
```

`larger()` returns a reference to the caller's own variable, so the call can even appear on the **left** of `=`: it's an lvalue! This is safe because `x` and `y` outlive the call.

6 Wrapping up

6.1 Summary

- Every expression has a **type** and a **value category**. An **lvalue** is an identifiable object; an **rvalue** is just a value.
- A **reference** (`int& ref{x};`) is another name for an existing object. It must be initialized and can't be reseated.
- A non-const reference binds only to modifiable lvalues; a **const reference** binds to anything.
- **Pass by reference** avoids copies and lets a function change its argument. **Pass by const reference** avoids copies without allowing changes.
- Pass fundamental types by value, class types by const reference.
- Never return (or keep) a reference to an object that is about to be destroyed: **dangling references** are undefined behaviour.

6.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Value categories (lvalues and rvalues)
2. Pass by const lvalue reference, and when to pass by value instead
3. `std::string_view`
4. Static local variables

Next up, Chapter 08: pointers, the other way to refer to an object.